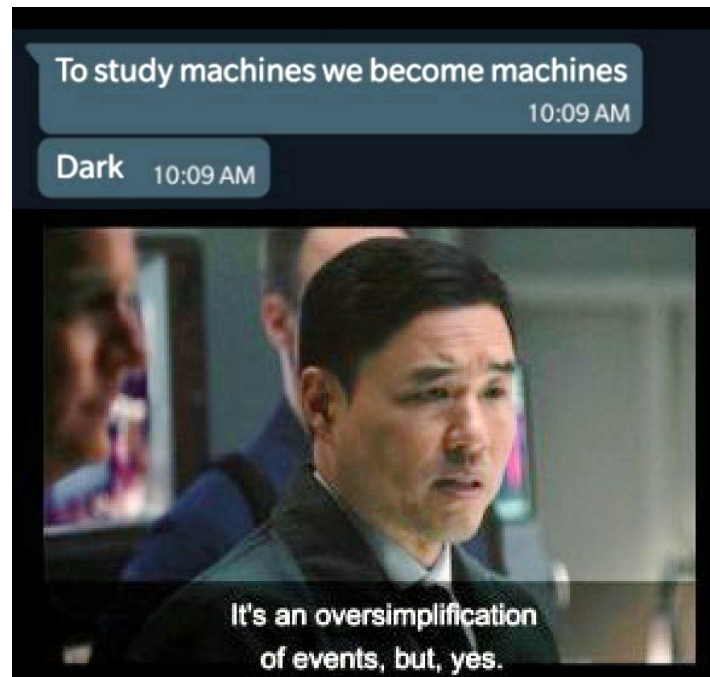
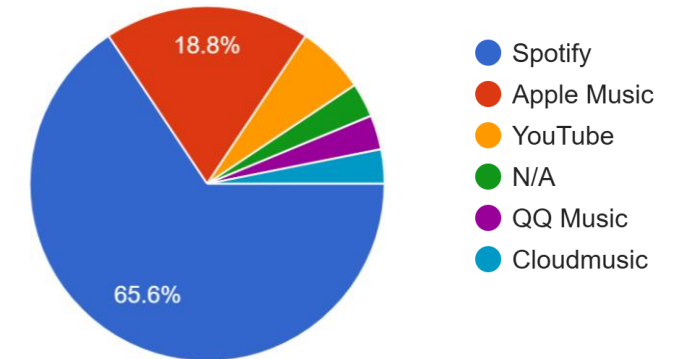


Unit 2: Computability



Preferred Music service?

32 responses



Sec 19: Th 4:30-5:30pm 2153 GGBL
IA: Anthony Chalhoub

Agenda



- ▶ Decision Problems and Languages
- ▶ DFAs
- ▶ Turing Machines
- ▶ Decidability

Decision Problems & Languages



Decision Problems and Languages

- ▶ **Definition:** A **decision problem** is a problem that has a 'yes' or 'no' answer
 - ▶ Example: “Is it possible to get from A to B in under 10 miles of travel?”
- ▶ **Definition:** The **language** of a decision problem is the **set of all ‘yes’ instances** to that problem

$$L_{canTravel} = \{(A, B): \text{There is a path to get from } A \text{ to } B \text{ in less than 10 miles}\}$$

- ▶ We say $w = (A, B) \in L_{canTravel}$ if and only if there is a path from A to B in less than 10 miles
- ▶ A language is a **set**, so all set operations are valid, for instance:

$$\overline{L_{canTravel}} = \{(A, B): \text{There is **no** path to get from } A \text{ to } B \text{ in less than 10 miles}\}$$

Wait! Computers can't read your inputs!

- ▶ To make an input computer readable, we need to **encode** it as a *string*
- ▶ But it can't be *any* string, we specify the **alphabet** (Σ) - a **finite** set of characters allowed to be used in the string
 - ▶ Example: $\{0, 1\}$, $\{0, \dots, 9\}$, $\{a, b, c\}$, set of ASCII characters
- ▶ We denote this encoding with $\langle \rangle$ braces
 - ▶ The encoding must be **unique** for any object being encoded
 - ▶ A string must be **finite** in length
- ▶ Examples:
 - ▶ The encoding of an integer could be itself as a string: $k = 376 \rightarrow \langle k \rangle = 376$
 - ▶ We stress that this is the string with **characters** 3, 7, 6
 - ▶ The encoding of a C++ program could be a string of its **source code**

Regular Expressions/String Notations for EECS 376

- ▶ ε is a **symbol** used to represent a string of 0 length (Don't get confused with $\emptyset$!)
- ▶ We use the characters in the alphabet, and special characters (), +, *, 'power', and |
 - ▶ The parentheses () mean to treat whatever is inside **as a single clause**
 - ▶ The kleene star * means to look for **zero or more** of the clause on the left
 - ▶ $(ab)^*$ matches $\varepsilon, ab, ababababab$, etc
 - ▶ Σ^* means any (finite-length) string (including ε) that uses symbols in Σ
 - ▶ The plus + means to look for **one or more** of the clause on the left
 - ▶ $aa^+ = a^+$ matches $a, aa, aaa, \dots$, but *not* ε
 - ▶ The 'power' means to look for **exactly** power of the clauses on the left
 - ▶ j^4 matches $jjjj$ only
 - ▶ The bar | means look for either the clause on the left **or** the clause on the right
 - ▶ $(CSE)|(ECE)$ matches 'ECE' or 'CSE'



Infinitely many strings vs Infinite string

- ▶ Give some example of strings that matches $a^*(ab)^*$
 - ▶ $\epsilon, a, ab, aaab, abab$
- ▶ How many strings are there that match this pattern?
 - ▶ Infinitely many
- ▶ Is it possible to have a string that matches $a^*(ab)^*$ that is infinite(-length)?
 - ▶ No! By definition, a string must be finite(-length)
 - ▶ Analogy: There are infinitely many natural numbers, and they grow arbitrarily large, but every individual natural number has some finite size

Let's Define Some Languages! (WS 2.1 #2)

- i. Define a reasonable (finite) alphabet Σ
- ii. Give the encoding (over the alphabet) of a representative input object
- iii. Define a language L for the corresponding decision problem
 - a) Given a binary string b , does it have an even number of 0s?
 - a) Does a given non-negative integer x , when written in base 10, have 3 as the last digit?

Let's Define Some Languages! (WS 2.1 #2)

- i. Define a reasonable (finite) alphabet Σ
- ii. Give the encoding (over the alphabet) of a representative input object
- iii. Define a language L for the corresponding decision problem
 - a) Given a binary string b , does it have an even number of 0s?
 - i. $\Sigma = \{0,1\}$
 - ii. For a binary string b , write it as a string of bits. For example, if b is the binary string "010", then $\langle b \rangle = 010$.
 - iii. $L = \{\langle b \rangle : b \text{ is a binary string with even number of 0s.}\}$
 - b) Does a given non-negative integer x , when written in base 10, have 3 as the last digit?

Let's Define Some Languages! (WS 2.1 #2)

- i. Define a reasonable (finite) alphabet Σ
 - ii. Give the encoding (over the alphabet) of a representative input object
 - iii. Define a language L for the corresponding decision problem
- a) Given a binary string b , does it have an even number of 0s?
- i. $\Sigma = \{0,1\}$
 - ii. For a binary string b , write it as a string of bits. For example, if b is the binary string "010", then $\langle b \rangle = 010$.
 - iii. $L = \{\langle b \rangle : b \text{ is a binary string with even number of 0s.}\}$
- b) Does a given non-negative integer x , when written in base 10, have 3 as the last digit?
- i. $\Sigma = \{0, \dots, 9\}$
 - ii. For an integer k , write it in base 10 in the usual way, as a string of digits. For example, if k is the integer forty-seven, then $\langle k \rangle = 47$.
 - iii. $L_{\text{EndsWith}_3} = \{\langle k \rangle : k \bmod 10 = 3\}$ OR $L_{\text{EndsWith}_3} = \{\langle k \rangle : k \text{ written in base 10 has 3 as the last digit}\}$

Deterministic Finite Automata

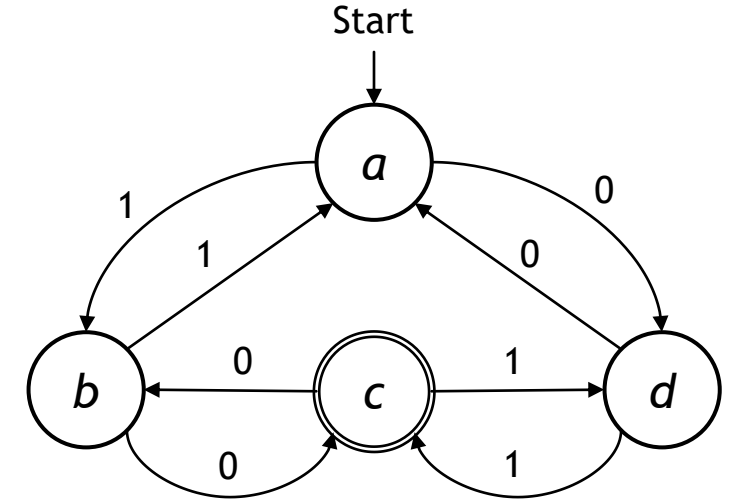
AKA Deterministic Finite-State Machine (DFSM) or Deterministic Finite-State Automata (DFSA)



Deterministic Finite Automata (DFA)

A DFA is defined by the five-tuple $(Q, \Sigma, q_0, F, \delta)$

- ▶ Q = set of states that the DFA has $Q = \{a, b, c, d\}$
- ▶ Σ = input alphabet that the DFA can read $\Sigma = \{0,1\}$
- ▶ q_0 = start state $q_0 = a$
- ▶ F = set of final (accept) state(s) $F = \{c\}$
- ▶ δ = transition function $\delta(\text{current}, \text{input}) = \text{next}$



State	Input	Next state
a	0	d
a	1	b
b	0	c
b	1	a
d	0	a
d	1	c

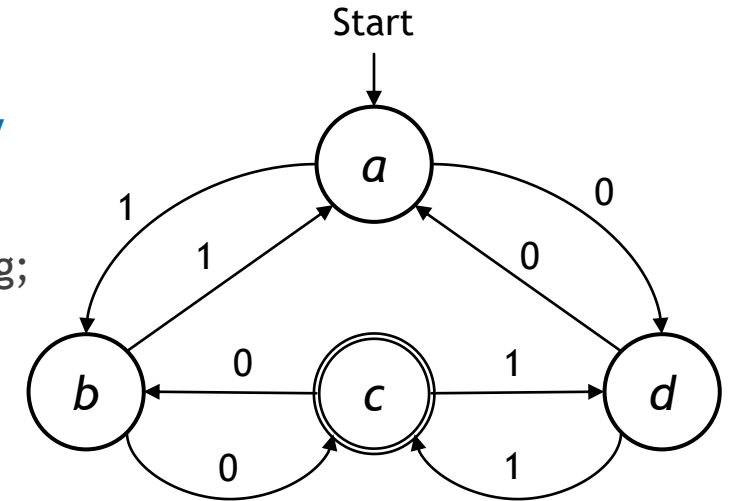


Common Mistakes

- Forget to indicate q_0 with "start→"
- Forget to indicate accept state with double circle
- Forget some transition functions for some non-final states

How DFA Works

- ▶ Given as input string s , DFA reads in **one character at a time exactly once**. After parsing the **entire** input string,
 - ▶ if it **ends** at an accept state $\in F$, we say the DFA **accepts** the input string;
 - ▶ otherwise, we say it **rejects** the input string
- ▶ For language L , we say that DFA D **decides** L if and only if D :
 - ▶ Correctly accepts all $w \in L$
 - ▶ Correctly rejects all $w \notin L$
- ▶ Determine if the DFA accept or reject "101010" and "101"
 - ▶ **Accept "101010"; reject "101"**
- ▶ Determine the language that the DFA decides
 - ▶ $L = \{\langle b \rangle: b \text{ is a binary string with odd 0s and odd 1s}\}$



State	Input	Next state
a	0	d
a	1	b
b	0	c
b	1	a
d	0	a
d	1	c

More Complicated DFAs

- ▶ **Fact:** Regular languages (languages decidable by DFAs) are closed under union and intersection
- ▶ On homework/exams, we might ask you to give DFAs that decide language in the form of

$$\{s \in \Sigma^*: s \text{ [Property A] **and** [Property B]}\}$$

or

$$\{s \in \Sigma^*: s \text{ [Property A] **or** [Property B]}\}$$

- ▶ Don't be panic! **Break it down!**
- ▶ Start with simpler DFAs that decides A and B separately, then "combine" them

Let's Define a DFA! (WS 2.2 #3)

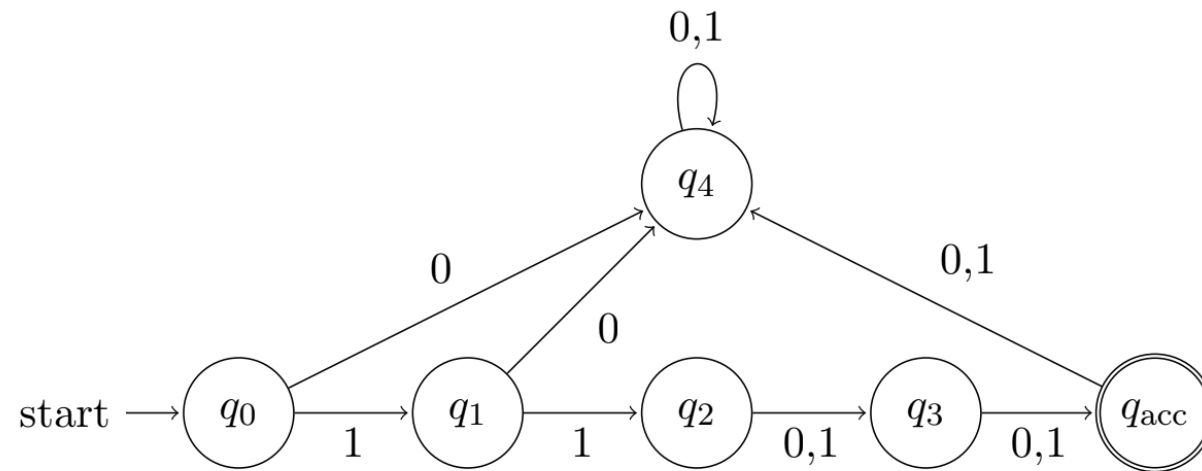
- ▶ True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}$

Let's Define a DFA! (WS 2.2 #3)

- True! There is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}$



DFA: Palindrome (WS 2.2 #4)

- ▶ Consider the following language over some alphabet with at least two elements

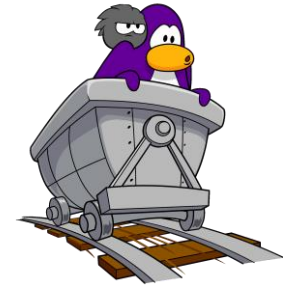
$$L = \{s \in \Sigma^* : s \text{ is a palindrome}\}$$

- ▶ True or False: there is a DFA that decides L .

DFA Impossibility Proof: Palindrome (WS 2.2 #4)

- ▶ Consider the following language over some alphabet with at least two elements

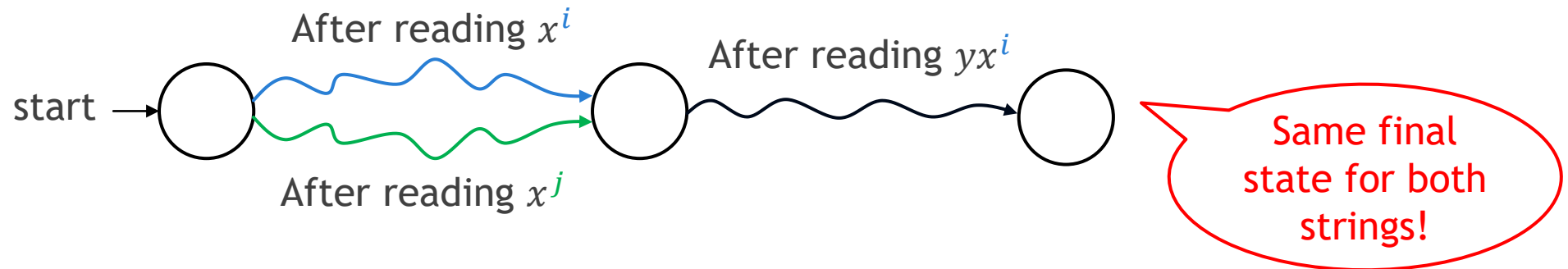
$$L = \{s \in \Sigma^* : s \text{ is a palindrome}\}$$



- ▶ **FALSE!** We claim that there is no DFA that decides the language
 - ▶ The key here is DFA only have **finite** states, so it can't "remember" or "count" an arbitrarily large part of an input string (Note: This is NOT saying that the input string can be infinite in length!)
- ▶ Suppose for contradiction that a DFA D decides L
 - ▶ By definition, D must have some finite number of states n
- ▶ Consider the input string x^{n+1} , where $x \in \Sigma$ is an arbitrary symbol in the alphabet
- ▶ By the **Pigeonhole principle**, there is some state in D that is repeated as D processes x^{n+1}
 - ▶ Consequently, there must be some distinct indices $0 \leq i < j \leq n + 1$, where D is in the same state after processing x^i and x^j

DFA Impossibility Proof: Palindrome, cont. (WS 2.2 #4)

- ▶ Now, consider another symbol $y \in \Sigma$, where $y \neq x$ (this is why we required $|\Sigma| \geq 2$)
- ▶ Let $s_1 = x^i y x^i$ and $s_2 = x^j y x^i$. Note that s_2 is not a palindrome because $i \neq j$
- ▶ However, D is in the same state after processing x^i and x^j , and the suffixes of s_1 and s_2 are both $y x^i$, so D must give the same answer on both s_1 and s_2 (either accept or reject)



- ▶ If D accepts, it **answers incorrectly** on s_2
- ▶ If D rejects, it **answers incorrectly** on s_1
- ▶ Therefore, D is not a DFA that decides L . This completes the contradiction.

Intermission

10 Tips to Succeed in EECS 376 (Part 4)

7

Small Examples



When tackling a problem or trying to prove a statement, work through **small examples** first (e.g., a single node in a graph, a two-element list). This will help **clarify the problem** and might **reveal patterns** or edge cases you hadn't considered.

8

Math Hygiene



Adopt rigorous mathematical practices. Use **precise terminology**, ensure each statement is well-formed, and verify that every step in your proofs **follows logically** from the previous one.

Turing Machines



Definition and Representation

- ▶ We define a Turing machine as the 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$
 - ▶ Q = finite set of states
 - ▶ Σ = **input** alphabet
 - ▶ Γ = **tape** alphabet ($\Gamma \supseteq \Sigma \cup \{\perp\}$, where $\perp \notin \Sigma$ is the blank symbol)
 - ▶ δ = transition function
 - ▶ q_0 = start state
 - ▶ q_{accept} = accept state(s)
 - ▶ q_{reject} = reject state(s)
- ▶ Turing machines can be represented with **state diagrams** or **pseudocode**
- ▶ The language of a TM M , denoted as $L(M)$ is the set of strings in which M **accepts**

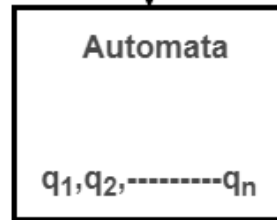
Finite Automata vs Turing Machines

Finite Automata

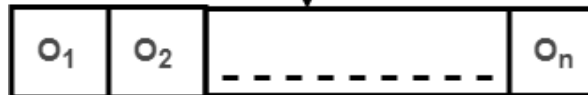
Finite tape with finite input alphabet



Read in one direction*



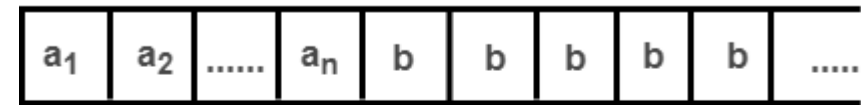
Finite number of states
Has a start state
Has accept states
Has no reject states



Halts after reading
all the inputs

Turing Machine

Infinite tape with finite input alphabet
and special symbols



Read or write in
both direction



Finite number of states
Has a start state
Has accept states
Has reject states

Halts after reaching an
accept/ reject state

Otherwise, we say it
loops indefinitely

*The head of a *two-way automata* can move in both directions, which is out of the scope of this class

Decidability



Decidability, Recognizability, and Turing Machines

- ▶ For a language L , we say Turing machine M **decides** L if:
 - ▶ (1) For all $x \in L$, M accepts x
 - ▶ (2) For all $x \notin L$, M rejects x
 - ▶ (3) And M **halts on all input** Q: Why do we not have this requirement for DFAs? **DFA halts on all inputs!**
 - ▶ We call M a **decider** of L
 - ▶ (1) and (2) together imply (3)
- ▶ **Definition:** Language L is **decidable** if and only if there exists a TM that decides L
 - ▶ We can prove that a language is decidable by giving a TM that decides it
 - ▶ **Reminder:** TM is computationally equivalent to computer programs, so "give a TM" = "give an algorithm" (except possible runtime analysis becomes halting analysis)
- ▶ For a language A , we say Turing machine M **recognizes** A if:
 - ▶ For all $x \in A$, M accepts x
 - ▶ For all $x \notin A$, M **does not accept** x (this could mean reject or loop!)

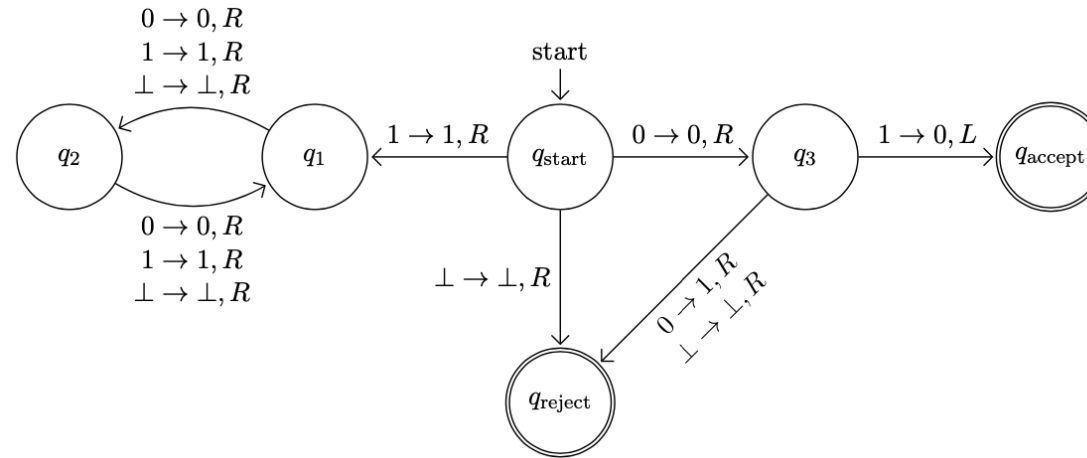
Decidability and Set Operations

- ▶ Since languages are *sets*, we can think about different operations on them. For instance, the class of decidable languages is closed under
 - ▶ **Set complement:** If L is decidable, then $\bar{L}$ is decidable
 - ▶ Proof idea: Run the TM that decides L , negate the decision
 - ▶ **Set union:** If A and B are both decidable, then $A \cup B$ is decidable
 - ▶ Proof idea: Run the TMs that decides A and B , accept if at least one of them accepts and reject otherwise
 - ▶ **Set intersection:** If A and B are both decidable, then $A \cap B$ is decidable
 - ▶ Proof idea: Run the TMs that decides A and B , accept if both of them accept and reject otherwise
- ▶ Combining the three, we can show that the class is closed under symmetric difference
- ▶ More: See "Decidability potpourri" or "-ability potpourri" in the addendum folder



Good exercise for
cheatsheet prep!

TM Example



- What language over $\Sigma = \{0,1\}$ does this TM decide, if any?
 - None. Observe that if the input strings start with 1, the TM will always loop. In other words, it fails to halt on input of form $1(0|1)^*$

Decidability Practice (WS 2.4 #1)

Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

Decidability Practice (WS 2.4 #1)

Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

Solution: $A \triangle B$ is decidable in **all** cases. We show this by describing a TM D that decides $A \triangle B$, using Turing machines M_A, M_B that respectively decide A, B (which exist by hypothesis). Specifically, $D(x)$ runs both $M_A(x)$ and $M_B(x)$, and accepts if exactly one of them accepts; otherwise, it rejects.

$x \in A \triangle B$

$x \notin A \triangle B$

Therefore, D decides $A \triangle B$, by definition.